

Slides:
tinyurl.com
/28rw2db3



Programming Like it's 1975

Nathan Jones





Slides:
tinyurl.com
/28rw2db3



What can you do with a 4-bit computer?

The logo for Supercon. 6, featuring the word 'Supercon.' in a large, white, sans-serif font, followed by a small yellow circle containing the number '6'. The logo is centered on a solid black rectangular background.

Supercon.⁶

Badge Hacking Awards Ceremony, Supercon 2022

Slides:
tinyurl.com
/28rw2db3

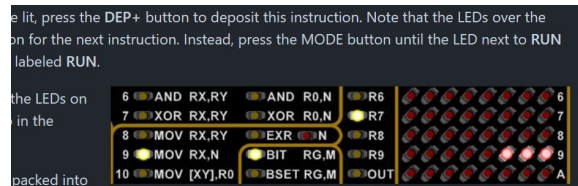


Agenda

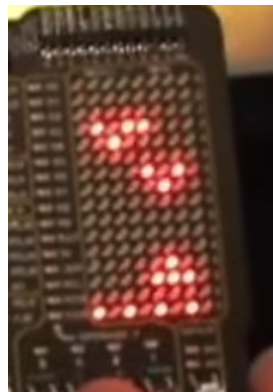
Hack-a-day Tutorial

Running the examples

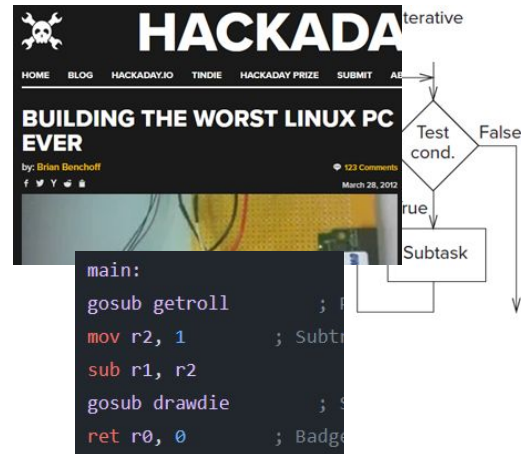
Advanced Topics and
Building Your Own Project



"Tertis"



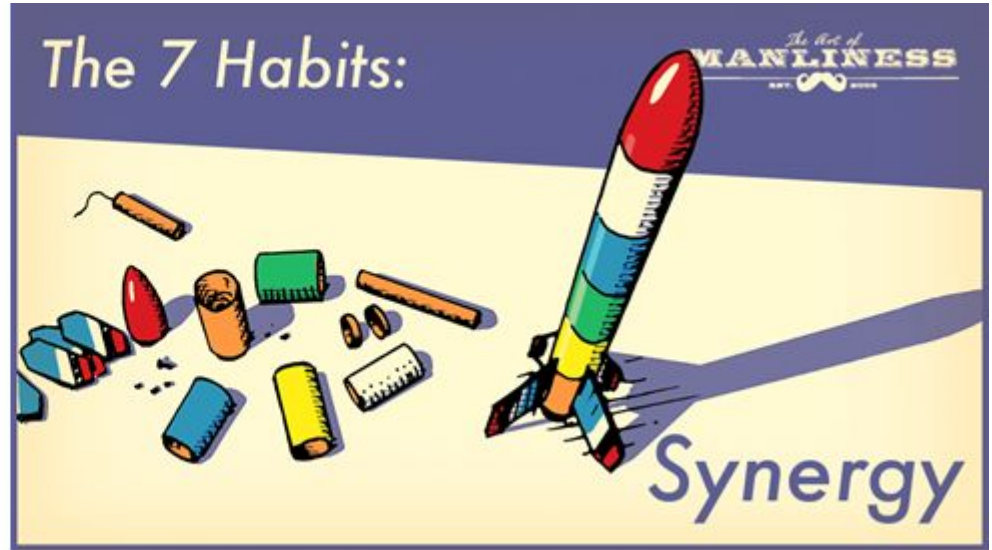
"Demon Attack"



Slides:
tinyurl.com
/28rw2db3



How can we maximize learning?



Slides:
tinyurl.com
/28rw2db3



How can we maximize learning?

- If you know/learn something, *everyone* around your table should know that same thing.
- If someone asks you for help, *their* problem becomes *your* problem (help them out!).
- If you get stuck on something, *ask for help*. (And then your problem has become both of your problems!)
- etc

Slides:
tinyurl.com
/28rw2db3



Get to know the folks at your table!

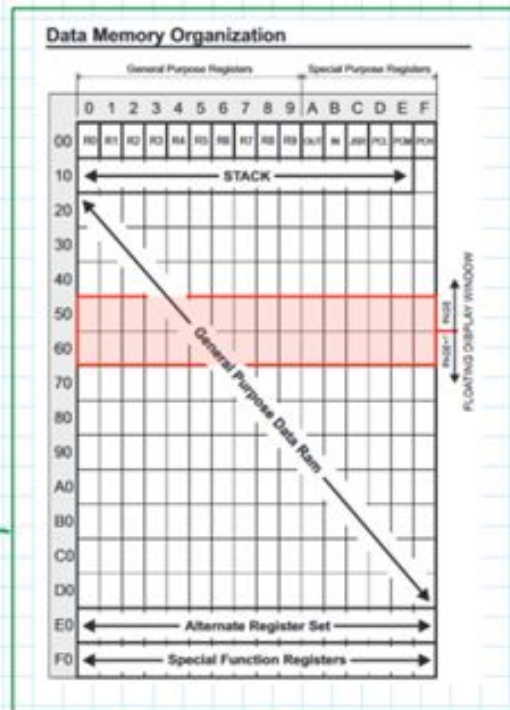
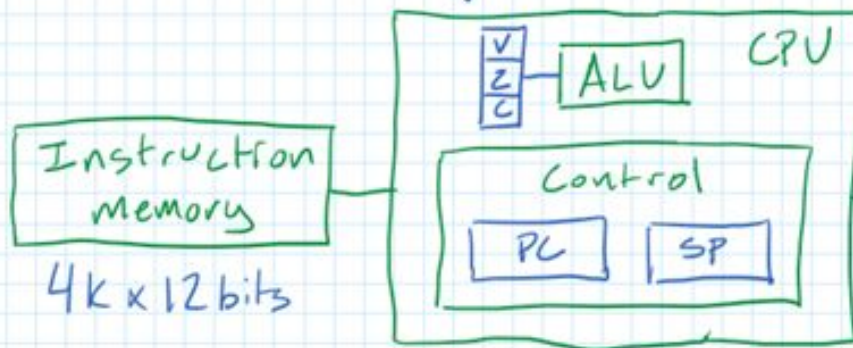
Ask them, "What inspired you to get into tech?"

Slides:
tinyurl.com
/28rw2db3



The Voja4 ISA

ADD RX,RY	CP R0,N
ADC RX,RY	ADD R0,N
SUB RX,RY	INC RY
SBB RX,RY	DEC RY
OR RX,RY	DSZ RY
AND RX,RY	OR R0,N
XOR RX,RY	AND R0,N
MOV RX,RY	XOR R0,N
MOV RX,NN	EXR N
MOV [XY],R0	BIT RG,M
MOV R0,[XY]	BSET RG,M
MOV [NN],R0	BCLR RG,M
MOV R0,[NN]	BTG RG,M
MOV PC,NN	RRC RY
JR NN	RET R0,N
	SKIP F,M



256 x 4 bits

Slides:
tinyurl.com
/28rw2db3



Tutorial!

<https://tinyurl.com/y3h7397f>



The screenshot shows the GitHub interface for the repository 'Hack-a-Day / 2022-Supercon6-Badge-Tools'. The 'Files' tab is selected, showing a directory structure with folders like 'assembler', 'emulator', 'examples', 'firmware', and 'manuals'. The 'tutorial' directory is expanded, and the file '0.Basics.md' is highlighted with a red circle. The file path is '2022-Supercon6-Badge-Tools / tutorial / 0.Basics.md'. Below the file list, there is a section for 'Add guided tutorial' by user 'MS3FGX'. The file '0.Basics.md' is shown in 'Preview' mode, displaying the title 'In the Beginning...' and the start of the text: 'Imagine it's the summer of 1975. A warm breeze blows t...'. The file statistics show '60 lines (34 loc) · 5.17 KB'.

Slides:
tinyurl.com
/28rw2db3

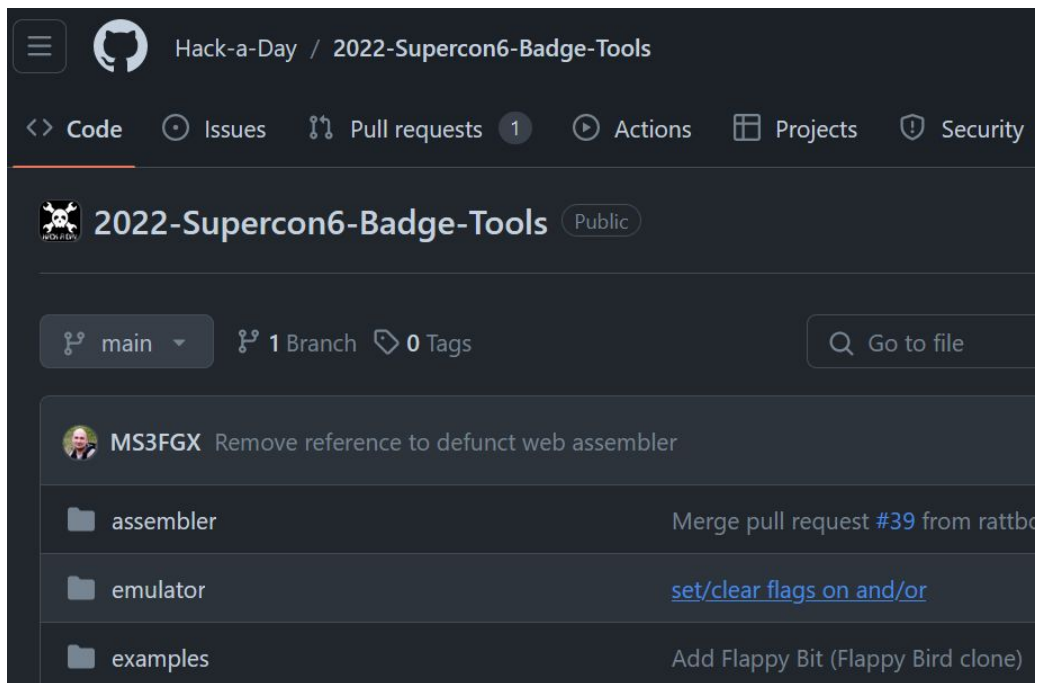


Running the Examples

Slides:
tinyurl.com
/28rw2db3



1. Clone "2022-Supercon-Badge-Tools"

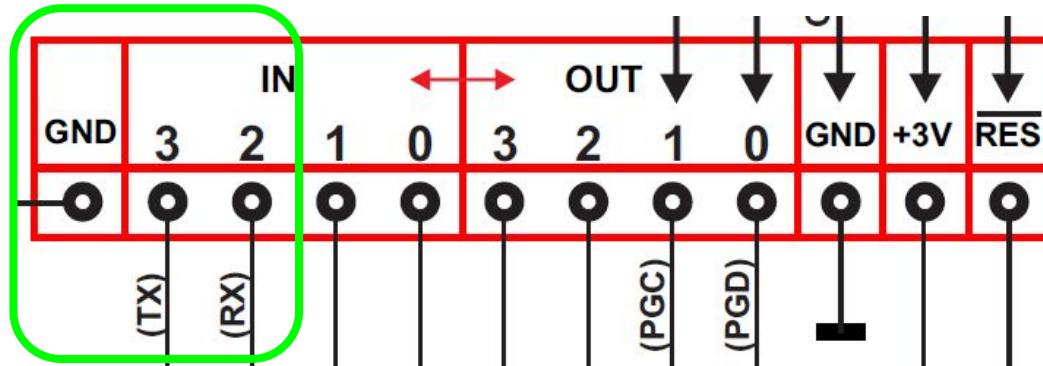




2. Connect a USB-to-UART adapter to the exp. port

Here are the rules for serial **Save/Load**:

- Serial Port settings: **9600, N, 8, 1** (not affected by **SFR** setting in **SerCtrl** register)
- Hardware considerations: **3V CMOS** Logic levels (**Tx/Rx** on **I/O connector** only)
- **Tx/Rx** on **SAO connector** cannot be used for program saving or loading



Slides:
tinyurl.com
/28rw2db3



3. Assemble your code

Option 1: Use the provided makefile

Option 2: Run the assembler yourself

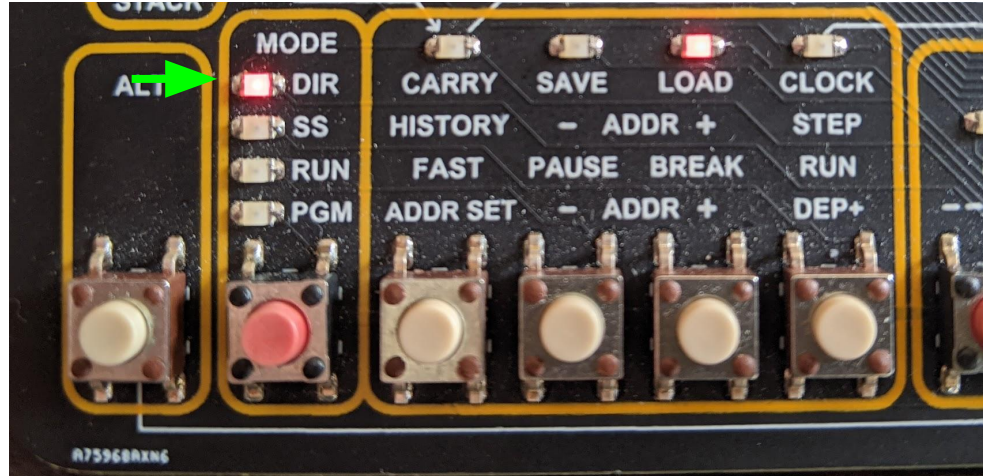
A terminal window with a dark background and light-colored text. The window title is 'nathancharlesjones@WPNBD'. The prompt is 'nathancharlesjones@WPNBDHJFGDC93: /mnt/c/Users/nathan.jones2/Documents/GitHub/2022-Supercon6-Badge-Tools/examples/martinling-conway-life\$'. The user has entered 'make life.hex'. The output shows the path to the assembler script and the version number. Below this, the contents of the assembly file are shown, starting with a comment about Conway's Game of Life.

```
nathancharlesjones@WPNBDHJFGDC93: /mnt/c/Users/nathan.jones2/Documents/GitHub/2022-Supercon6-Badge-Tools/examples/martinling-conway-life$  
make life.hex  
../../assembler/assemble.py life.asm  
Supercon.6 Badge Assembler version 1.0  
  
; Conway's Game of Life  
; for Hackaday Berlin 2023 / Supercon 2022 badg
```



4. Put your badge into program load mode

- a. Press MODE until you're in DIR mode.
- b. OPTIONAL: Save your current program to a portion of Flash memory
 - i. Press ALT+[Operand Y] to select the section of flash memory to be written
 - ii. Press ALT+SAVE
- c. Press LOAD.



Slides:
tinyurl.com
/28rw2db3



5. Send .hex file out over UART

Linux

```
nathancharlesjones@WPNBDHJFGDC93:/mnt/c/Users/nath  
Tools/examples/martinling-conway-life$ make flash
```

Windows
(update PORT first!)

```
Windows PowerShell  
PS C:\Users\nathan.jones2> cd .\Documents\GitHub\2022-Supercon6-Badge-Tools\  
PS C:\Users\nathan.jones2\Documents\GitHub\2022-Supercon6-Badge-Tools> python '.\software  
Windows Tools\win_flash.py' .\examples\martinling-conway-life\life.hex
```


Slides:
tinyurl.com
/28rw2db3



Find an example and run it!

Slides:
tinyurl.com
/28rw2db3



Building Your Own Project

Slides:
tinyurl.com
/28rw2db3



Brainstorm ideas with your tablemates!

Slides:
tinyurl.com
/28rw2db3



Helpful Topics Before You Begin

- About the badge
 - The rest of the Voja4 instructions
 - Synthetic instructions
 - SFRs
 - Other modes and features of the badge
 - Contents of the manuals
- About assembly programming
 - Idioms: loops, conditionals, subroutine calls, pointers, etc
 - Debugging tips
 - Flowcharts for reading/writing assembly

Slides:
tinyurl.com
/28rw2db3

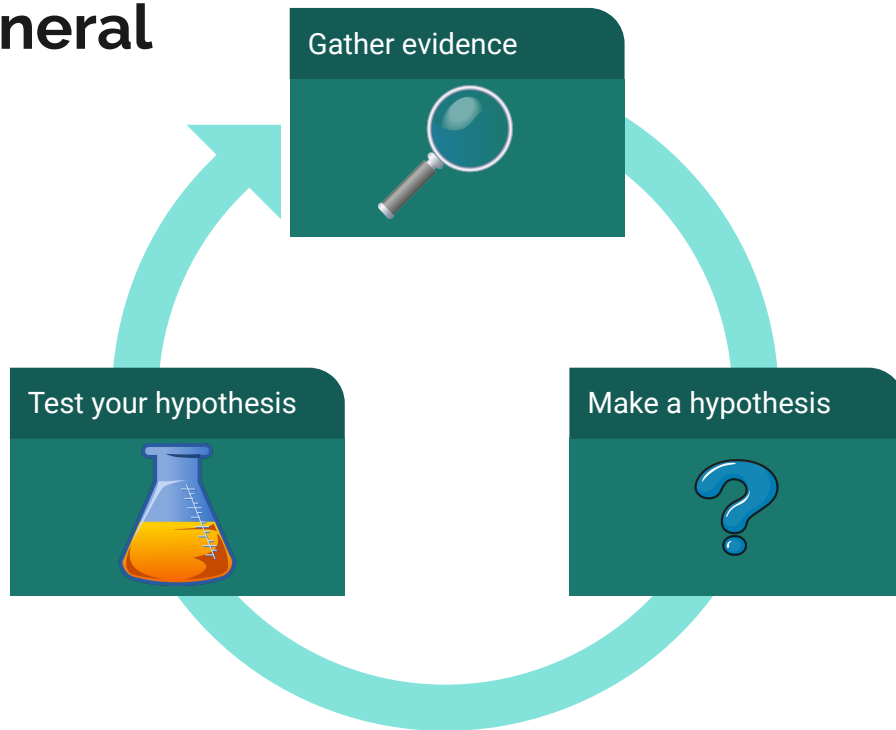


Debugging Tips

Slides:
tinyurl.com
/28rw2db3



General

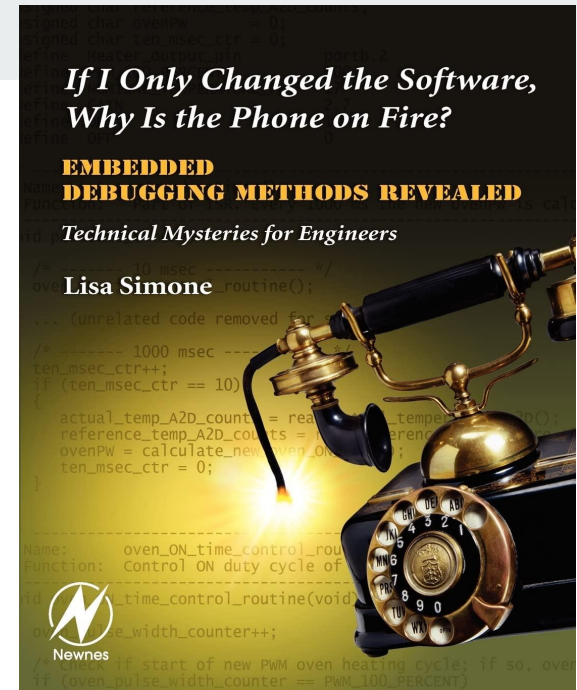
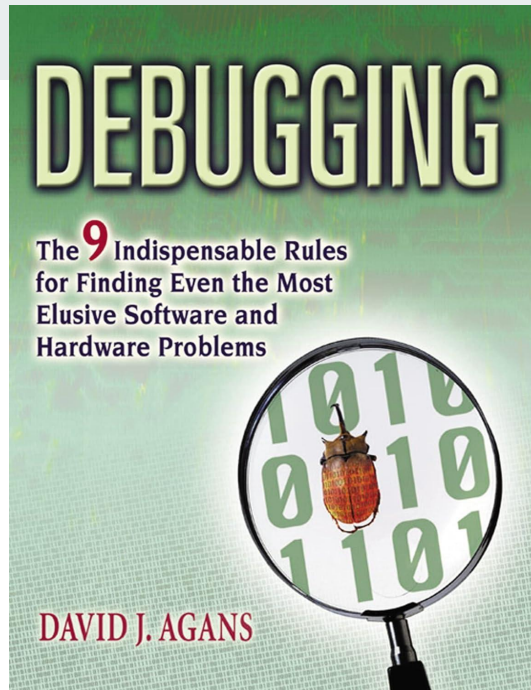


- Change **ONE** thing at a time
- Write down what you're doing
- Read the manual (especially the parts that don't make sense!)
- Draw a sketch
- **Take a break**
- After you've fixed your bug:
 - Scan your code for similar errors
 - Add this error to a checklist

Slides:
tinyurl.com
/28rw2db3



Resources



- ["How to Debug"](#) by John Regehr
- "Debugging" by David Agans ([book](#) | [website](#))
- ["If I Only Changed the Software, Why is the Phone on Fire?"](#) by Lisa Simone
- ["Creating Chaos and Hardfaults"](#) by Elecia White
- ["Debugging"](#) lesson by Nathan Jones (part of EE375, "Computer Architecture")

Slides:
tinyurl.com
/28rw2db3



Badge-specific

Button Commands

MODE	COMMAND KEYS				OPCODE	OPER X	OPER Y
SS ALT	History	- Addr	Addr +	Step	Opcode	Oper X	Oper Y
	Enter History submode	Decrement Program Memory Address	Increment Program Memory Address	Execute one instruction	Instruction Opcode (bits 11-8)	Direct Operand X or Opcode (bits 7-4)	Direct Operand Y (bits 3-0)
	Toggle Carry Flag	Reset Program Memory Address to 0x000	Preset Program Memory Address to the last word used	Address set from Opcode, Operand X and Operand Y	User Sync select	Processor Clock select	Display Page select
RUN ALT	Fast	Pause	Break	Run	Opcode	Oper X	Oper Y
	On/Off Toggle 10x Faster Clock and Sync	Program Execution Pause / Resume	Terminate Program Execution	RUN Program From Program Memory	—	—	—
					User Sync select	Processor Clock select	Display Page select



Badge-specific: Implementing "breakpoints"

01

Assemble code with "JR -1" at desired breakpoint

```
217 gosub SETUP
218 gosub WAIT
219 jr -1
```

02

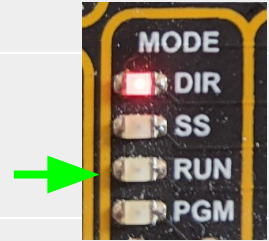
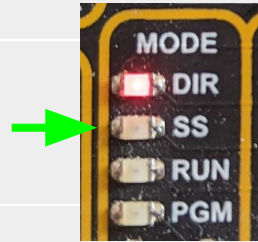
Run program until "JR -1" is reached

03

Press BREAK then switch to "Single-step" mode

04

Enter address *after* "JR -1" and press ALT+SAVE



	History	- Addr	Addr +	Step
ALT SS	Enter History submode	Decrement Program Memory Address	Increment Program Memory Address	Execute one instruction
	Toggle Carry Flag	Reset Program Memory Address to 0x000	Preset Program Memory Address to the last word used	Address set from Opcode, Operand X and Operand Y

Slides:
tinyurl.com
/28rw2db3

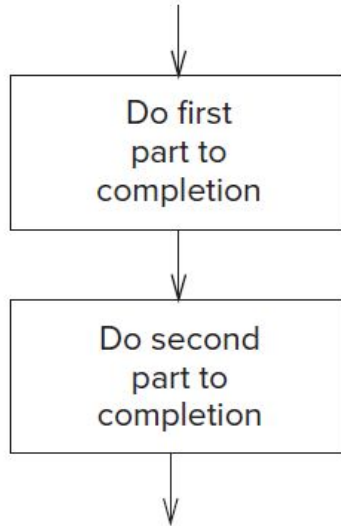


Flowcharts for Reading/Writing Assembly

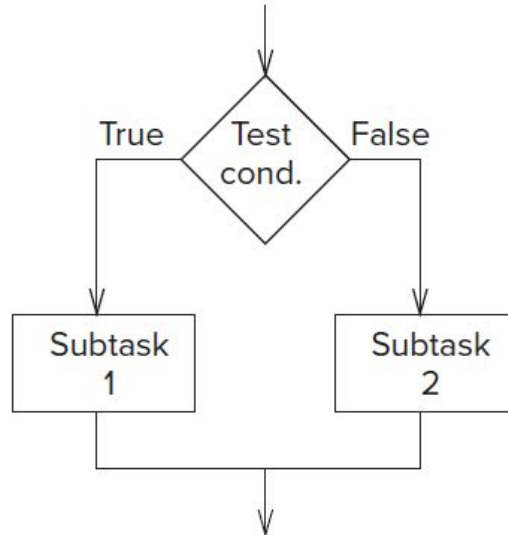


Flowchart Elements

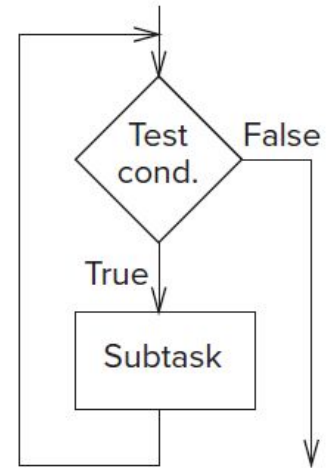
Sequential



Conditional



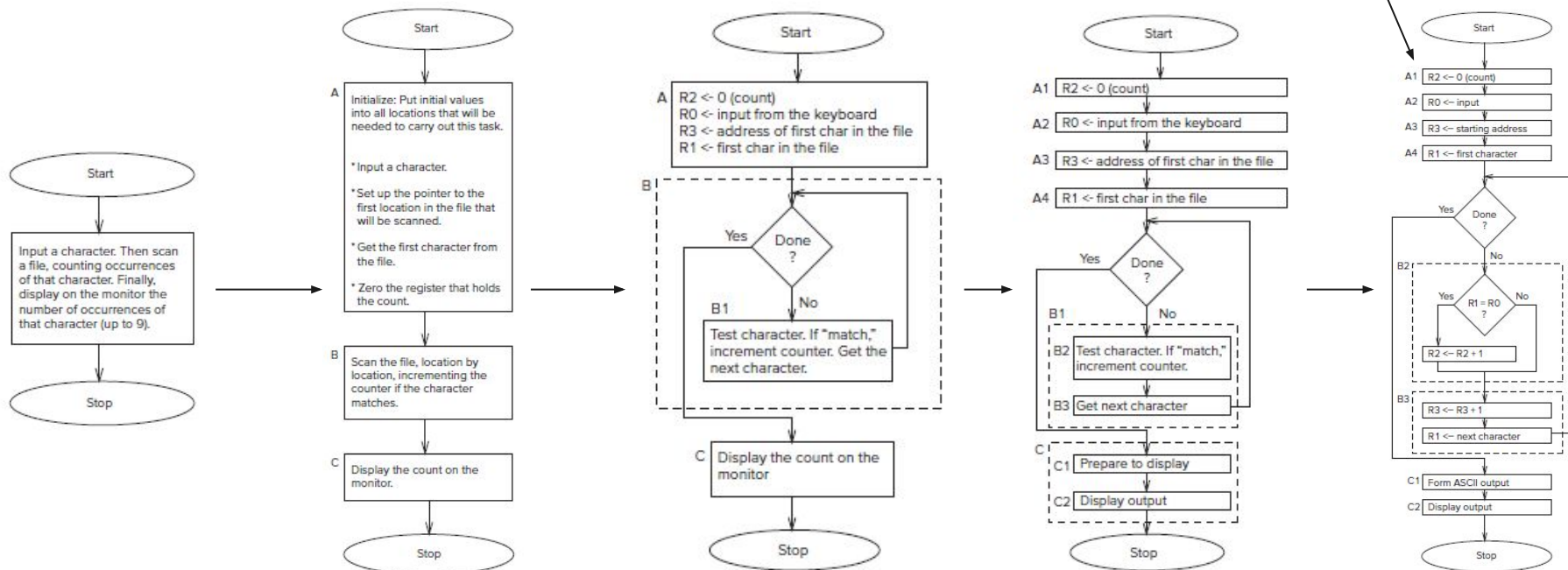
Iterative





Can be implemented in machine code

Writing Code Using Structured Decomposition



Slides:
tinyurl.com
/28rw2db3



Reading Code by Creating a Flowchart

```
foo:  
mov r0, [0xff]  
  
cp r0, 7  
skip nc, 1  
  
jr -4  
  
cp r0, 0  
skip nz, 1  
  
jr -7  
  
mov r1, r0  
  
ret r0, 0
```

Slides:
tinyurl.com
/28rw2db3



Reading Code by Creating a Flowchart

1. The start of the program starts a sequential block.

foo:

```
mov r0, [0xff]
```

```
cp r0, 7
```

```
skip nc, 1
```

```
jr -4
```

```
cp r0, 0
```

```
skip nz, 1
```

```
jr -7
```

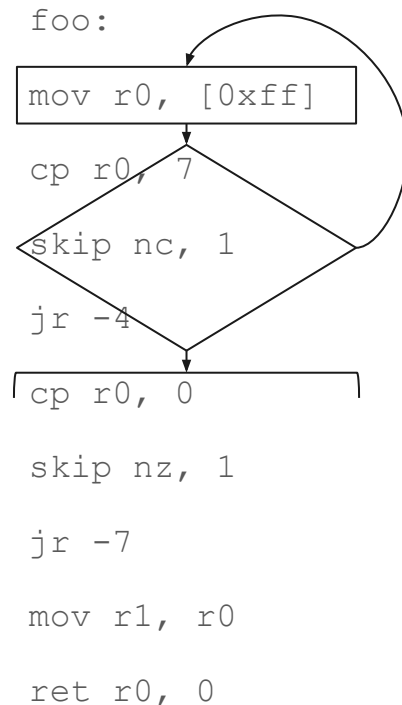
```
mov r1, r0
```

```
ret r0, 0
```




Reading Code by Creating a Flowchart

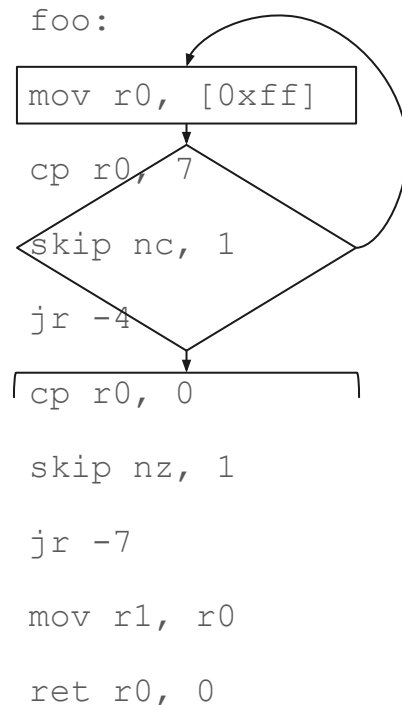
1. The start of the program starts a sequential block.
2. A jump instruction:
 - a. ends a sequential block (with the instruction prior to it),
 - b. is, itself, a selection block, and
 - c. starts a new sequential block (with the instruction following it).





Reading Code by Creating a Flowchart

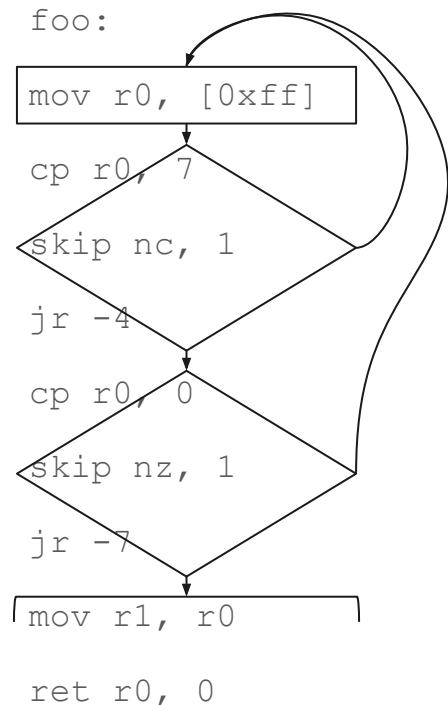
1. The start of the program starts a sequential block.
2. A jump instruction:
 - a. ends a sequential block (with the instruction prior to it),
 - b. is, itself, a selection block, and
 - c. starts a new sequential block (with the instruction following it).
3. A jump *target* starts a sequential block (and ends the previous one).





Reading Code by Creating a Flowchart

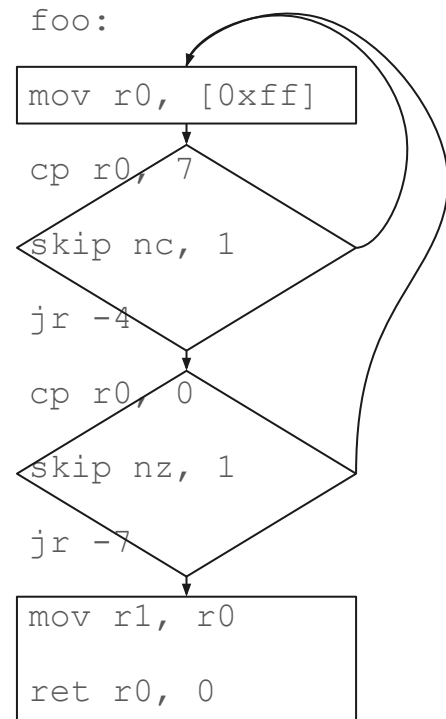
1. The start of the program starts a sequential block.
2. A jump instruction:
 - a. ends a sequential block (with the instruction prior to it),
 - b. is, itself, a selection block, and
 - c. starts a new sequential block (with the instruction following it).
3. A jump *target* starts a sequential block (and ends the previous one).





Reading Code by Creating a Flowchart

1. The start of the program starts a sequential block.
2. A jump instruction:
 - a. ends a sequential block (with the instruction prior to it),
 - b. is, itself, a selection block, and
 - c. starts a new sequential block (with the instruction following it).
3. A jump *target* starts a sequential block (and ends the previous one).
4. The end of the program ends a sequential block.





Reading Code by Creating a Flowchart

Get [0xFF] again if [0xFF] <= 7

Get [0xFF] again if [0xFF] == 0

Put [0xFF] in R1

foo:

```
mov r0, [0xff]
```

```
cp r0, 7
```

```
skip nc, 1
```

```
jr -4
```

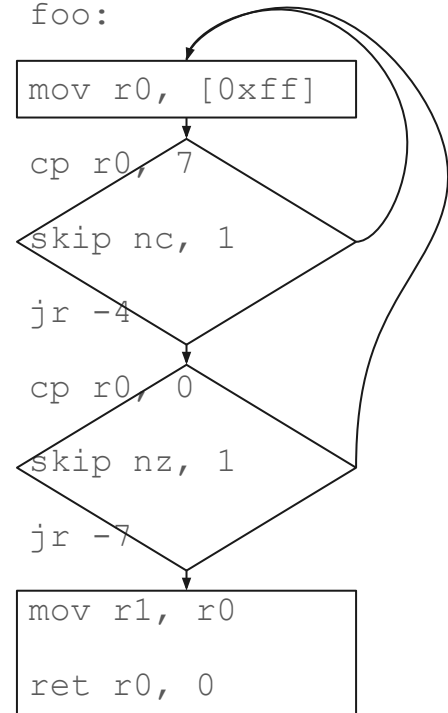
```
cp r0, 0
```

```
skip nz, 1
```

```
jr -7
```

```
mov r1, r0
```

```
ret r0, 0
```





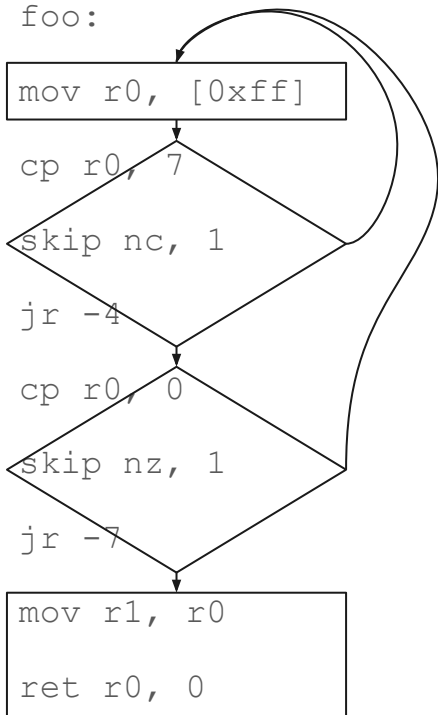
Reading Code by Creating a Flowchart

```
do
{
    R1 = *(0xFF);
} while (R1 >= 7) || (R1 == 0);
```

Get [0xFF] again if [0xFF] <= 7

Get [0xFF] again if [0xFF] == 0

Put [0xFF] in R1





Reading Code by Creating a Flowchart

Get [0xFF]

Get [0xFF] again if [0xFF] <= 7

Generate a random number from [1,6]

Get [0xFF] again if [0xFF] == 0

Put [0xFF] in R1

foo:

mov r0, [0xff]

cp r0, 7

skip nc, 1

jr -4

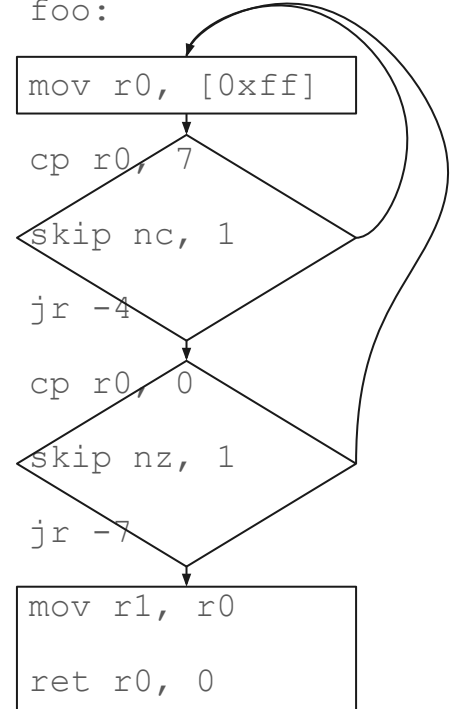
cp r0, 0

skip nz, 1

jr -7

mov r1, r0

ret r0, 0



Slides:
tinyurl.com
/28rw2db3



Let's build!



Slides:
tinyurl.com
/28rw2db3



Concluding

Slides:
tinyurl.com
/28rw2db3



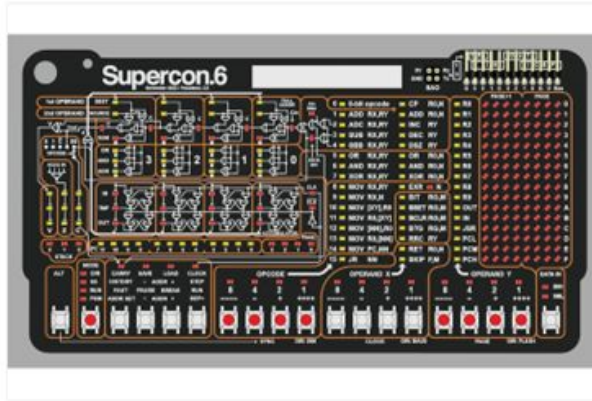
Buying More Voja4s

Supercon 2022 Badge (Voja4)

Badge from the 2022 Hackaday Supercon

Sold By 0xc0decafe

Wishlist Tweet Share Pin



\$70.00

Ask a Question

No shipping info available.
Set destination country to see options

Out of Stock

Sign up to get notified when this product is back in stock!

Desired Quantity*

1

Please message the seller directly if your desired quantity is very large.

<https://www.tindie.com/products/0xc0decafe/supercon-2022-badge-voja4/>

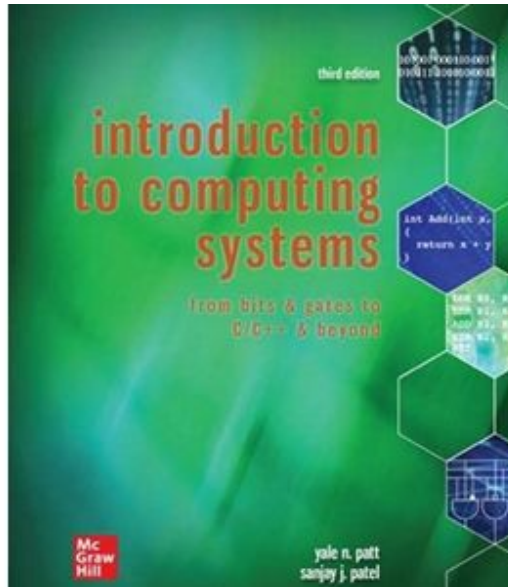
Slides:
tinyurl.com
/28rw2db3



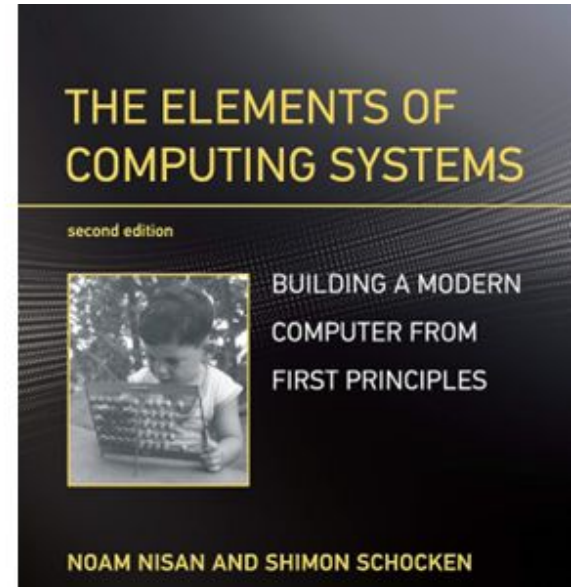
Going Further: Books/Courses



tinyurl.com/5n6f3zt6



tinyurl.com/39sa23fb
tinyurl.com/5398wrd5



tinyurl.com/mrpbnk48
nand2tetris.org

Slides:
tinyurl.com
/28rw2db3



Going Further: "Inside the Voja4"



<https://www.youtube.com/watch?v=cUcKpRigZts>



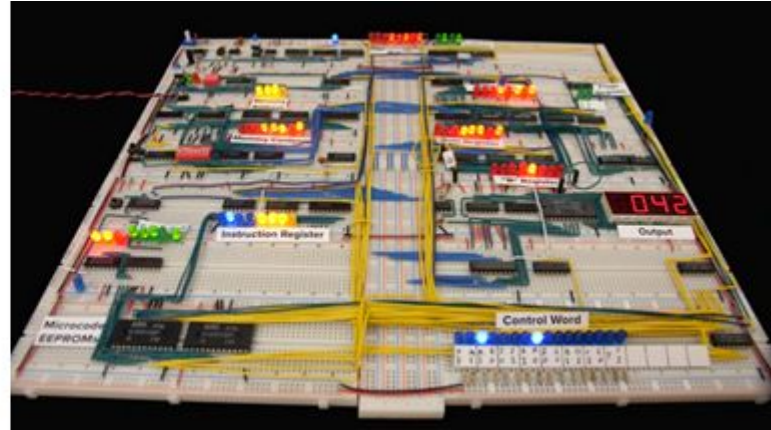
Going Further: Breadboard Computers

Nibbler 4-bit CPU (from BMOW)



bigmessowires.com/nibbler/

Ben Eater's 8-bit CPU



eater.net/8bit/kits

Slides:
tinyurl.com
/28rw2db3



**Thanks for coming and I hope
you enjoy the rest of the
conference!**
